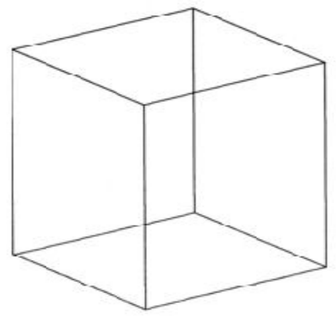
37. The Universal Machine

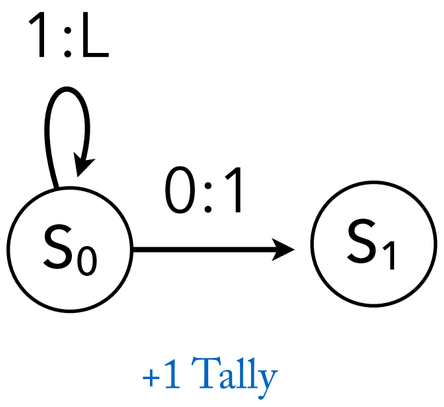
The Idea

The goal: Create a Turing Machine which can perform the task of any other Turing Machine. This is called a **Universal Turing Machine** (UTM).



universal machine

Step 1: Identify a method for translating a machine into a **code number**, which will function as the **program** for that machine.



+1 Tally

STATE	IN	OUT	NEXT
S_0	1	L	S_0
S_0	0	1	S_1
S_1	1	H	
S_1	0	H	

0, 1, 2, 0
0, 0, 1, 1
1, 1, 4, 0
1, 1, 4, 0

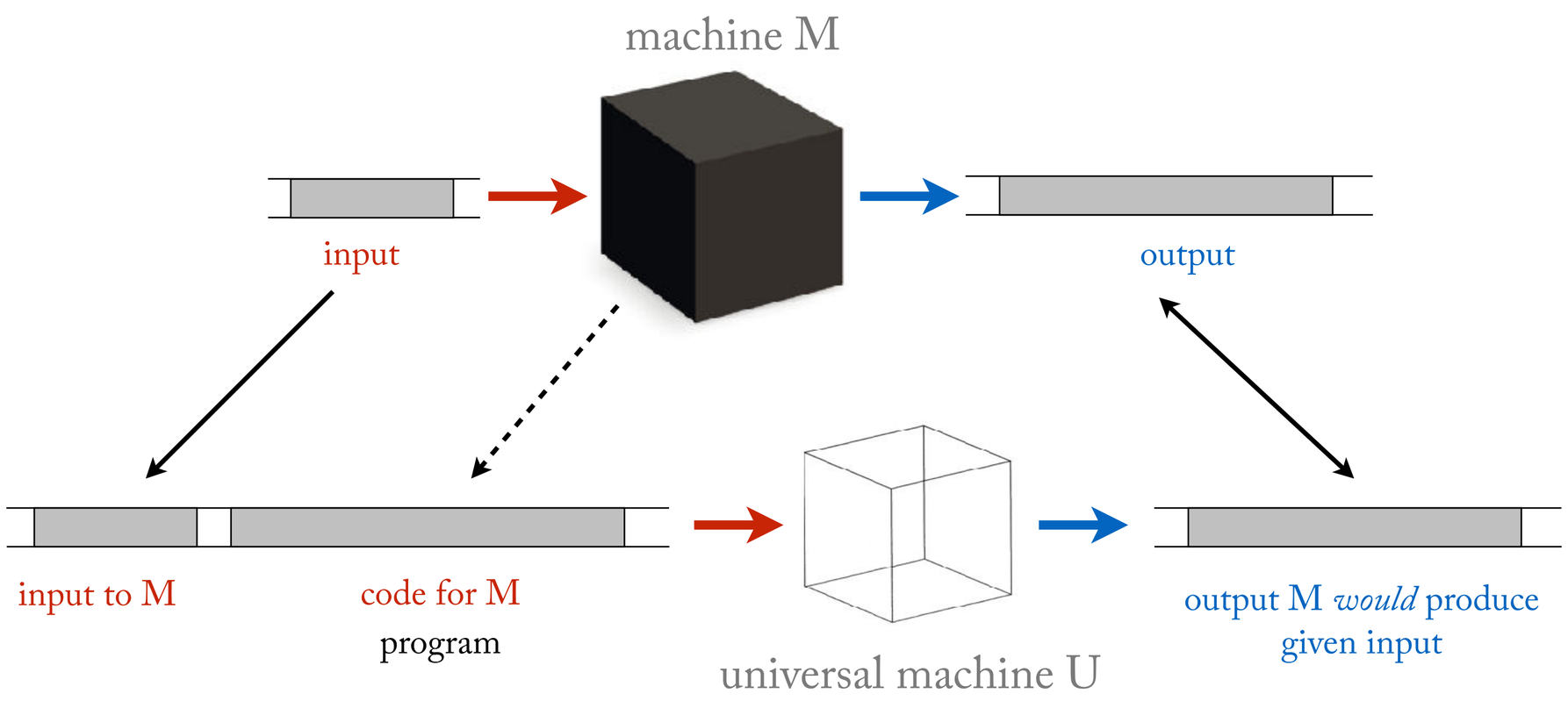
Turing machine → state table → numeral array

* 001011000 * 00001010 * 10101111000 * 10101111000 *

row 1 row 2 row 3 row 4

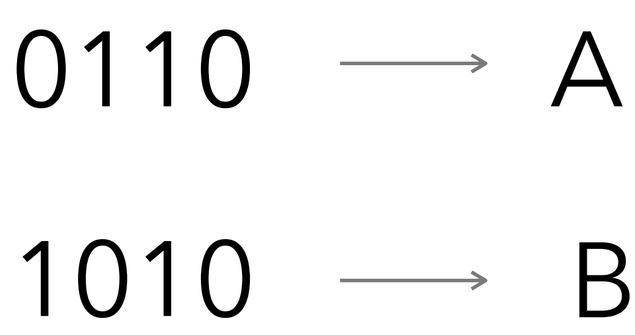
→ code number

Step 2: design a machine U which takes as input (i) any simulated machine M's code number, and (ii) M's input on a given occasion; U then produces the output which M would have produced in response to that input.



Markers

It's often useful to design an algorithm which makes use of more than two symbols. We can do this with 0's and 1's, by making *sequences* of 0's and 1's stand for additional symbols. This is more or less how your computer stores the 26+ letters of the English alphabet.

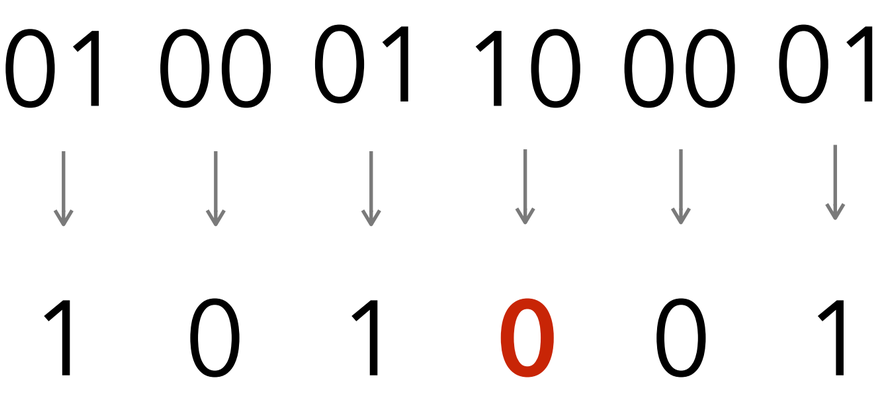


In particular, it is valuable to introduce a symbol which serves as a **marker** to indicate that a given symbol is in someway different from other symbols. Markers can be used to hold your place while computing, or two mark the end or beginning of a block.

A simple and useful scheme is to allow each *two* bits to represent a symbol. Then the the right hand bit will represent the symbol itself, and the left hand bit will indicate whether it is "marked".

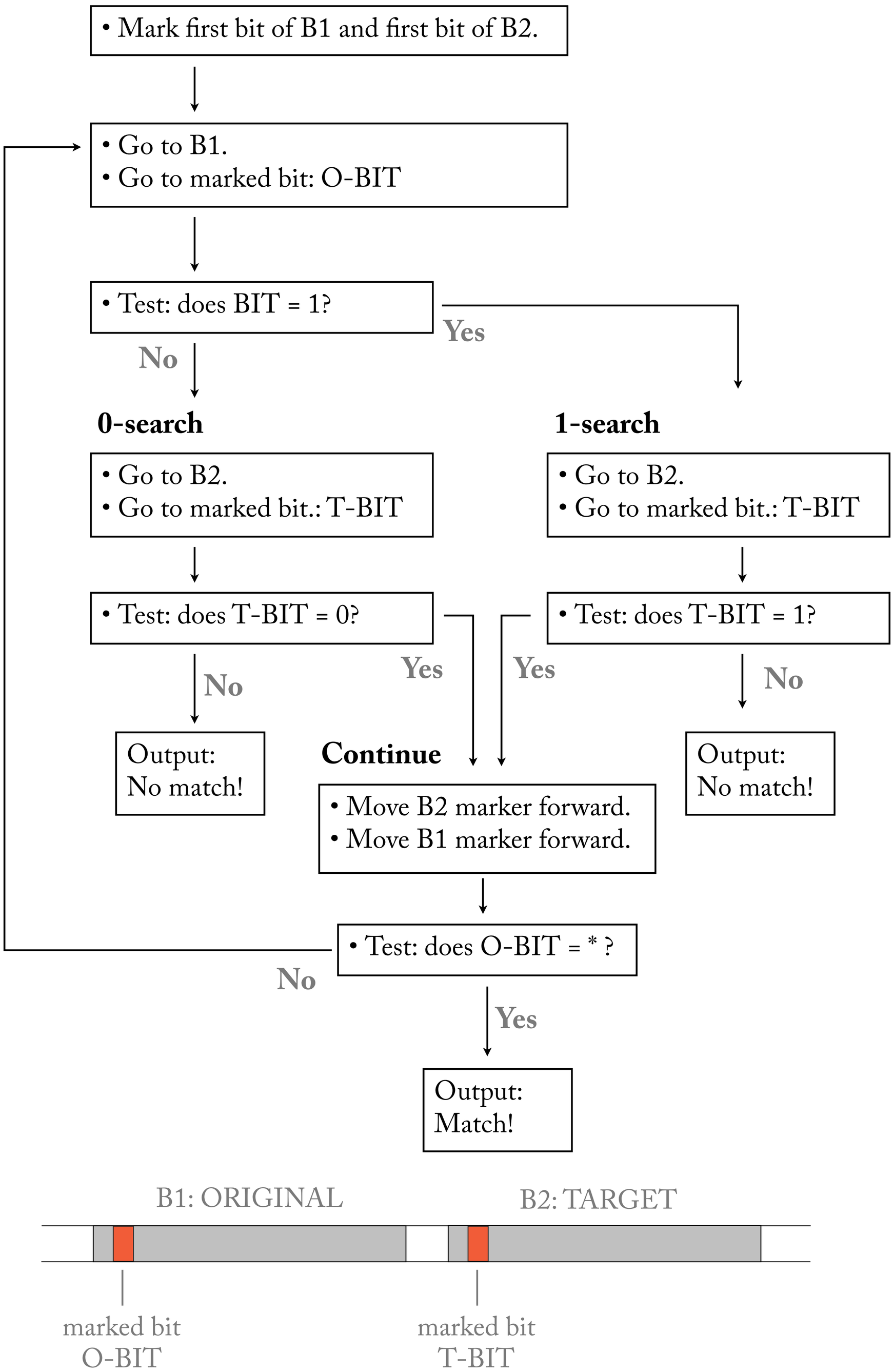


Now we can pick a marked symbol out of a crowd:



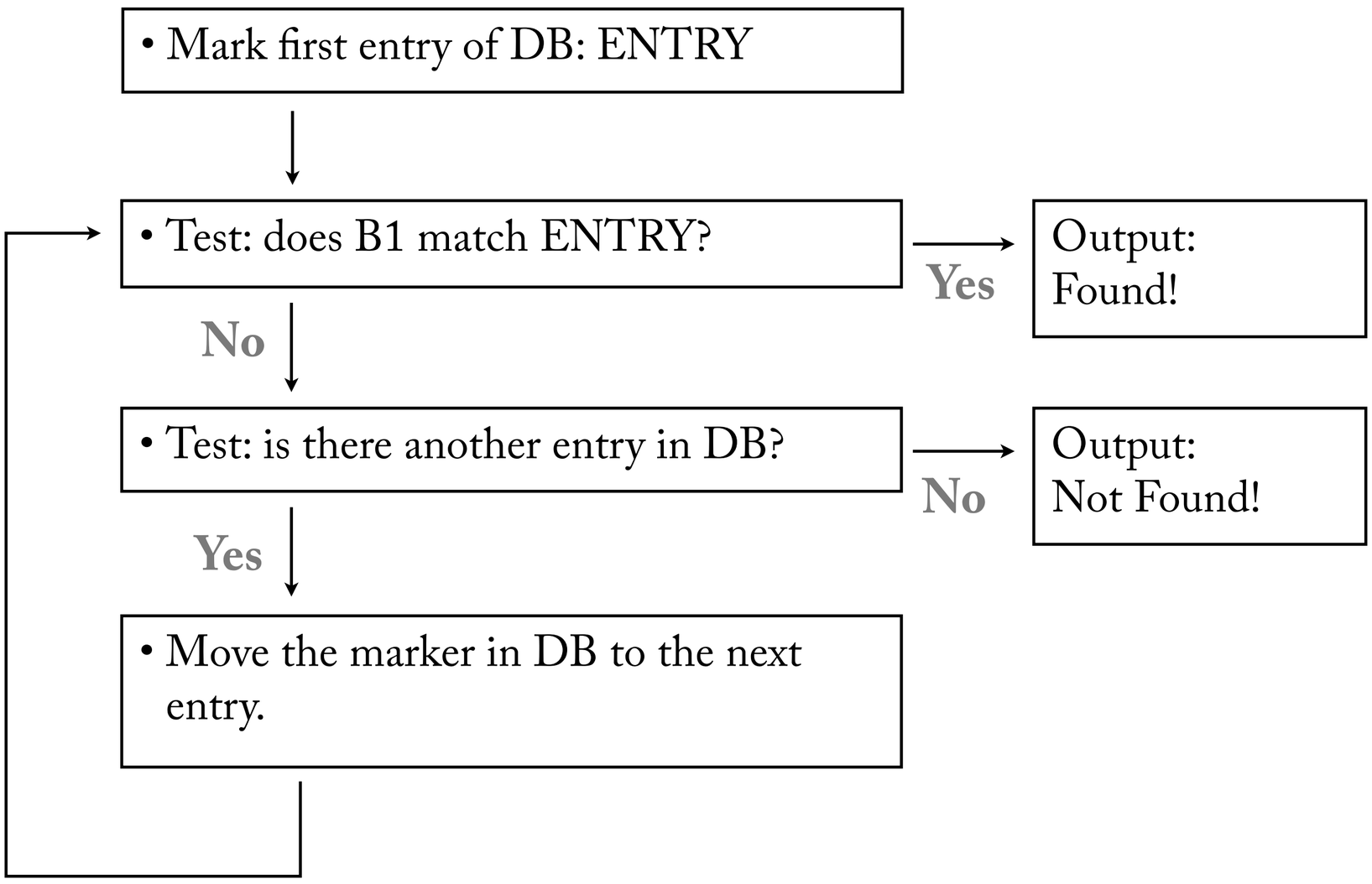
Matching

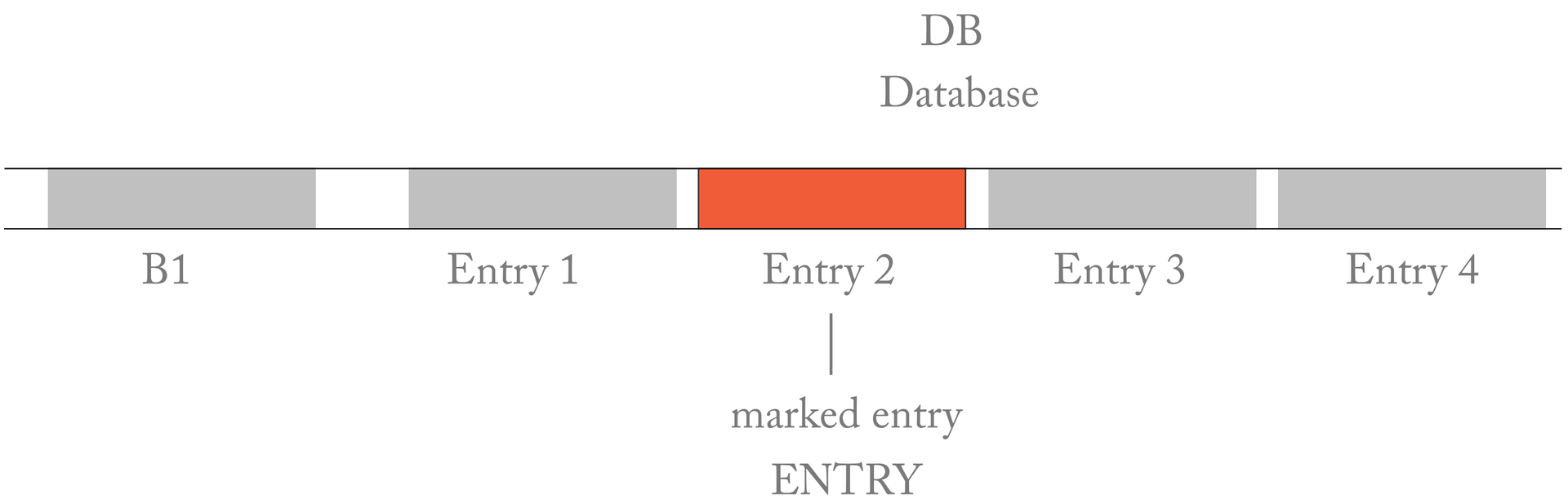
This algorithm compares an original string to a target string, and tests whether the two match or not.



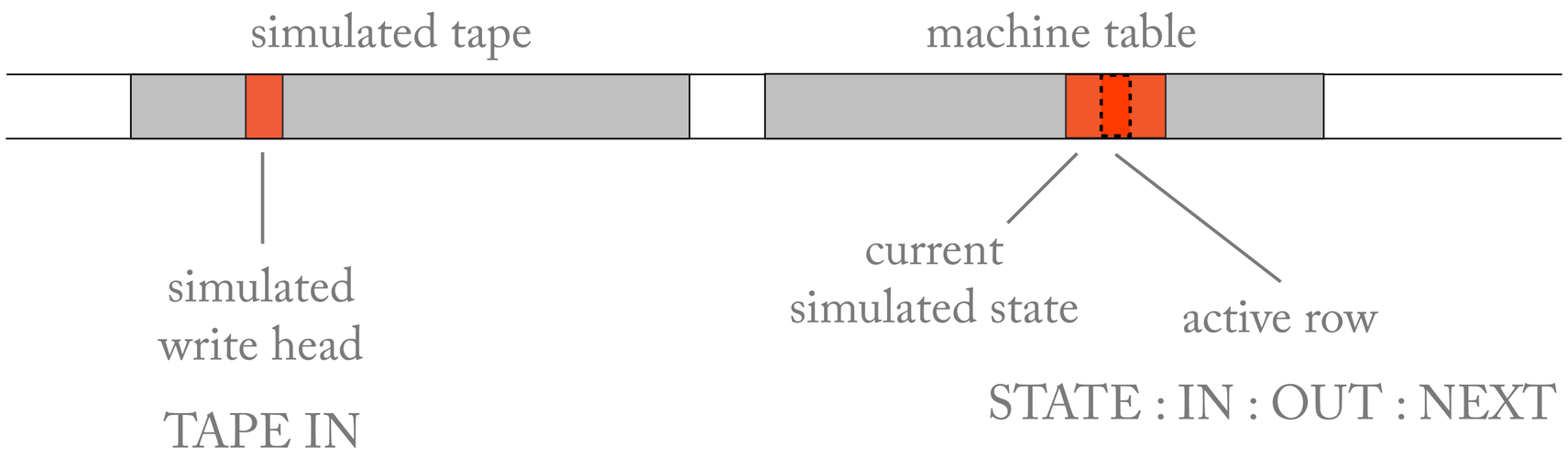
Search

This algorithm searches through a database to find an entry that matches some original sample. It outputs “Found!” or “Not Found!” depending on the result of the search.





Design for a Universal Turing Machine



Initialize + select first state
- Move the simulated write head to the first symbol of the simulated tape.
- Makes the first state of the machine table the current simulated state.

Initialize the simulated tape set the current state to S0.

Select current instruction
- Search the input column of the current simulated state for TAPE IN.
- Make that row the active row: STATE : IN : OUT : NEXT

Search for the current row based on the input from the simulated tape…

Check for final instruction
- Test: does OUT = Halt?

Yes → **Clean up**
- Clean up and halt.

No

Perform current instruction
- Perform OUT on the simulated tape, at the position of the simulated write head.

…make the appropriate simulated output: writing or moving on the simulated tape.

Select next state
- Search the state column of the machine table for NEXT.
- Make that state the current simulated state.

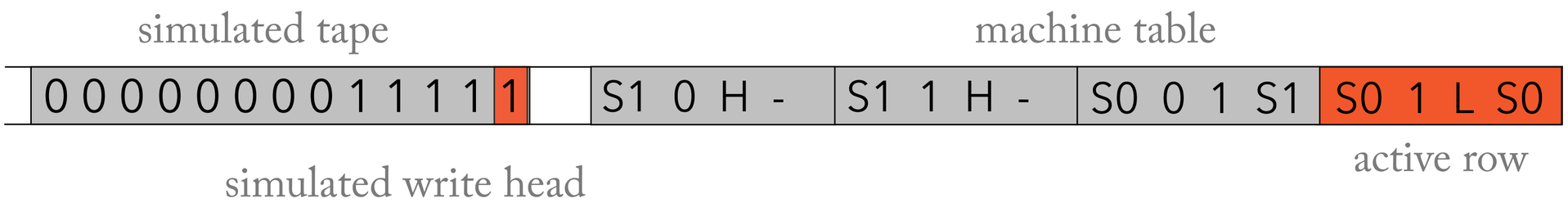
Find the next state named by the current row of the machine table, and make that the new current state.

Operation of a Universal Turing Machine

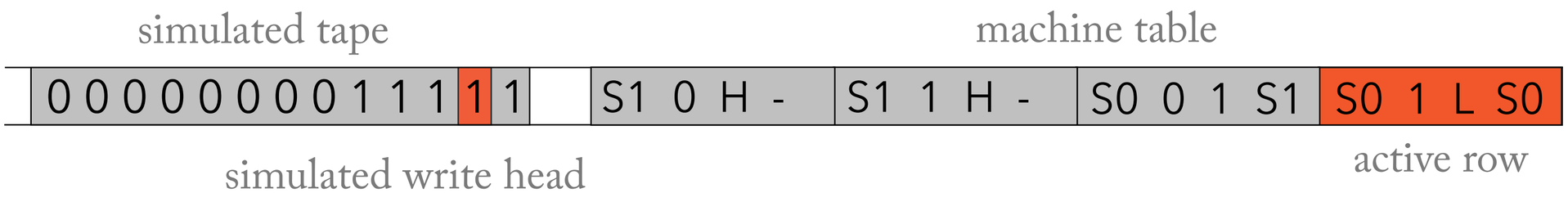
The following sequence illustrates the progress of the UTM, given the following machine table as its program.

STATE	IN	OUT	NEXT
S_0	1	L	S_0
S_0	0	1	S_1
S_1	1	H	
S_1	0	H	

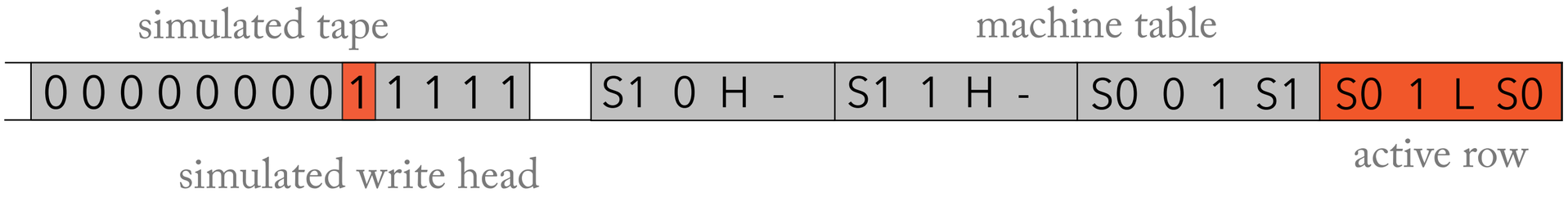
Initial state of the tape:



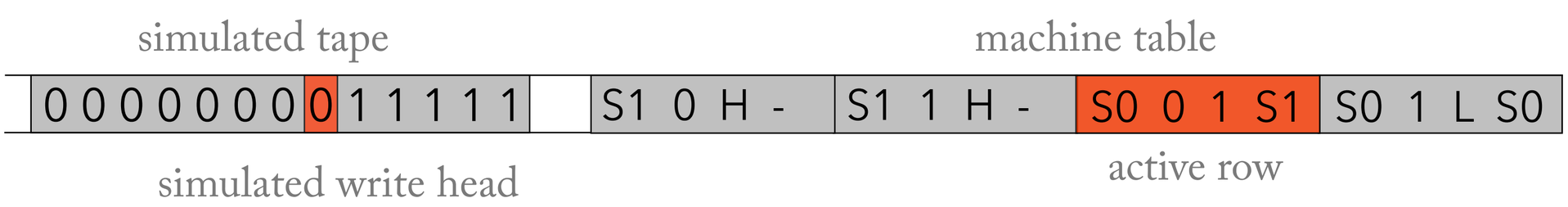
After 1 input/output cycle:



After 4 cycles:



After 5 cycles:



After 6 cycles:

